

Carbon Crunch — OCR Receipt Extraction: Technical Documentation

30 August 2026

1. APPROACH

- **Objective:** Build a confidence-aware OCR pipeline to extract structured data (store name, date, items, total amount) from 371 real-world receipt images and generate a financial summary.
- **Dataset Triage:** Classify each image into clean, blurry, skewed, or low_light using blur score (Laplacian variance), skew angle (Hough transform), and brightness metrics.
- **Preprocessing:** Bucket-specific cleanup using resize, denoise, deskew, CLAHE contrast enhancement, unsharp masking, and perspective correction for handheld photos.
- **OCR Engine:** EasyOCR as the primary CPU/GPU engine, with pytesseract as a secondary vote on clean images; word-level detections are grouped into lines by y-coordinate clustering.
- **Field Extraction:** Regex date parsing across multiple formats; keyword-anchored total extraction with reference-line exclusion for phone numbers, barcodes, and transaction IDs; OCR-artifact normalization; x-coordinate column clustering for item/price extraction.
- **Confidence Scoring:** $0.5 \times \text{OCR-confidence} + 0.3 \times \text{pattern-validity} + 0.2 \times \text{keyword-anchoring}$. Fields below 0.7 are flagged with specific reasons: missing_field, low_ocr_confidence, unparsable_format, or no_keyword_anchor.
- **Financial Summary:** Aggregate spend across all 371 receipts with fuzzy store-name matching using SequenceMatcher (0.85 threshold), reporting all-receipts and high-confidence-only totals.
- **Pipeline Orchestrator:** End-to-end runner with resume support and a --force flag for full reprocessing.

2. TOOLS USED

- Python 3.11; OpenCV (preprocessing); EasyOCR + pytesseract (OCR); NumPy/Pandas (data handling); regex (field extraction); difflib (fuzzy store matching).
- Development: local Windows CPU for early iterations; Kaggle Notebooks with GPU T4x2 for full 371-image pipeline runs, where EasyOCR is significantly faster.
- Version control: Git/GitHub for iterative fixes across multiple debugging rounds.

3. DATASET CHARACTERISTICS

- 371 receipt images (~143 MB): ~349 SROIE-style clean scanned receipts (Malaysian retail, consistent tabular layout) and ~22 handheld phone photos (variable lighting, skew, background clutter).
- No ground-truth annotations were provided; evaluation therefore relied on manual spot-checks and diagnostic scripts rather than automated precision/recall metrics.

4. RESULTS SUMMARY

- All 371 images processed successfully (0 failures).
- Average field confidence: store_name 0.66, date 0.65, total_amount 0.77.
- Financial summary: total spend across all receipts $\approx$ RM 25,374; high-confidence-only spend $\approx$ RM 14,422; 127 receipts flagged for manual review; 281 unique stores identified after fuzzy matching.
- 15 of 371 receipts (~4%) could not yield a total_amount due to severe OCR degradation, such as TOTAL being misread as IOTAL, or the total line missing entirely from OCR output.

5. CHALLENGES FACED

- OCR engine version differences: cv2.HoughLinesP output shape varied across OpenCV versions and initially caused unpacking crashes; fixed with a version-agnostic reshape.
- Regional number formatting: comma decimal separators such as 93,80 were initially misparsed as 9380; normalization now distinguishes decimal-comma from thousands-separator-comma.
- False-positive total extraction: barcodes, phone numbers, and transaction/terminal IDs were initially captured as totals; explicit reference-line exclusion and sanity-range ceilings were added.
- First-match vs rightmost-match bug: regex initially captured quantities such as 14 from TOTAL FOR 14 ITEMS 338.16; extraction now evaluates all matches and prefers the rightmost amount because amount columns are conventionally right-aligned.
- No ground truth: a targeted diagnostic script was used to manually categorize and debug extraction failures across a 24-receipt sample instead of relying on automated accuracy metrics.

6. IMPROVEMENTS / FUTURE WORK

- Add fuzzy/edit-distance keyword matching (e.g., Levenshtein distance) for OCR misreads such as TOTAL $\rightarrow$ IOTAL or TOTAI, addressing the current ~4% no-total failure rate.
- Fine-tune a receipt-specific OCR or layout-aware model such as LayoutLMv3 or Donut, particularly for the handheld-photo cluster.
- Create a manually labeled gold-standard subset of 30–40 receipts for proper precision/recall evaluation.
- Run GPU-based fine-tuning experiments specifically on the noisiest handheld-photo bucket.